

Introduction of DOM

Part 1: HTML to Browser Pipeline in DOM



When we write HTML, CSS, and JavaScript, the browser cannot directly understand them.

The browser follows a complete pipeline to convert your code into a visual webpage.

Understanding this pipeline helps you understand:

- DOM
- CSSOM
- Rendering
- Reflow
- Repaint
- Browser Performance

The Big Question

Suppose you write:

```
<!DOCTYPE html>

<html>

<head>

<title>My Website</title>

</head>

<body>

<h1>Hello World</h1>

</body>

</html>
```

How does this become a webpage?

The answer is:

HTML

↓

Parsing

↓

Tokenization

↓

DOM Tree

↓

CSSOM Tree

↓

Render Tree

↓

Layout

↓

Paint

↓

Screen

Complete Browser Pipeline

HTML File

↓

Parsing

↓

Tokenization

↓

DOM Tree

↓

CSS File

↓

Parsing

↓

CSSOM Tree

DOM + CSSOM

↓

Render Tree

↓

Layout

↓

Paint

↓

Screen

Real-Life Analogy

Imagine building a house.

HTML

Raw construction material.

Bricks

Doors

Windows

DOM Tree

Blueprint of the house.

Which room is connected to which room

CSSOM

Decoration instructions.

Wall colors

Furniture

Lighting

Render Tree

Final decorated house plan.

Layout

Measure room sizes.

Paint

Actually paint and build everything.

Why This Pipeline Matters

Every webpage you visit goes through this process.

Examples:

- Google
- Facebook
- Amazon
- YouTube

All browsers perform this pipeline before showing content.

Quick Revision

Step	Purpose
HTML	Raw document
Parsing	Reading code
Tokenization	Breaking into tokens
DOM Tree	HTML structure
CSSOM Tree	CSS structure
Render Tree	Combined visual structure
Layout	Calculate positions
Paint	Draw pixels

Part 2: Parsing

Parsing is the process where the browser reads the HTML document and understands its structure.

What is Parsing?

Parsing means:

| Reading code and converting it into a structure the browser can understand.

Suppose browser receives:

```
<h1>Hello</h1>
```

Browser cannot immediately display it.

First it must understand:

This is a heading element

Its content is "Hello"

This understanding process is called Parsing.

Parsing Flow

HTML Code

↓

Parser Reads Character by Character

↓

Creates Meaningful Pieces

↓

DOM Tree

Example

HTML:

```
<body>
```

```
<h1>Hello</h1>
```

`</body>`

Parser reads:

`<`

`body`

`>`

`<`

`h1`

`>`

Hello

`</h1>`

`</body>`

And understands the hierarchy.

Why Parsing is Important

Without parsing:

`<h1>Hello</h1>`

would simply be text.

Browser would not know:

- Tag names
- Nesting
- Parent-child relationships

Important Interview Point

The browser starts parsing HTML from top to bottom.

`<body>`

`<h1>Hello</h1>`

`<p>Paragraph</p>`

`</body>`

Order matters.

Quick Revision

Term	Meaning
Parsing	Reading and understanding HTML
Input	HTML code
Output	Structured representation
Direction	Top to Bottom

Part 3: Tokenization

Before building the DOM Tree, the browser converts HTML into tokens.

What is Tokenization?

Tokenization means:

| Breaking HTML into small understandable pieces called Tokens.

Example

HTML:

`<h1>Hello World</h1>`

Browser breaks it into:

Start Tag Token → `<h1>`

Text Token → `Hello World`

End Tag Token → `</h1>`

Visual Flow

`<h1>Hello</h1>`

↓ Tokenization

[Start Tag h1]

[Text Hello]

[End Tag h1]

Types of Tokens

1. Start Tag Token

`<div>`

Token:

Start Tag: div

2. End Tag Token

`</div>`

Token:

End Tag: div

3. Text Token

Hello

Token:

Text: Hello

4. Attribute Token

``

Tokens:

Tag: img

Attribute: src

Value: image.jpg

Why Tokenization Exists

Computers understand structured data better than raw text.

Instead of:

`<div class="box">Hello</div>`

Browser converts it into manageable pieces.

Quick Revision

Token Type	Example
Start Tag	<div>
End Tag	</div>
Text	Hello
Attribute	class
Value	box

Part 4: DOM Tree

This is one of the most important concepts in Web Development.

What is DOM?

DOM stands for:

Document Object Model

It is a tree-like representation of an HTML document.

Why DOM Exists?

JavaScript cannot directly manipulate HTML.

Instead:

HTML

↓

DOM

↓

JavaScript Manipulates DOM

Example

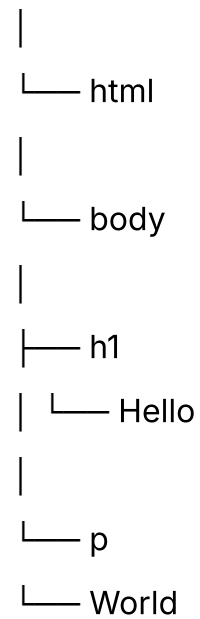
HTML:

<html>

```
<body>
<h1>Hello</h1>
<p>World</p>
</body>
</html>
```

DOM Tree:

Document



DOM Terminology

Parent Node

```
<body>
<h1>Hello</h1>
</body>
```

body is parent of h1.

Child Node

h1 is child of body.

Sibling Node

```
<h1>Hello</h1>
```

```
<p>World</p>
```

h1 and p are siblings.

DOM is a Live Structure

JavaScript can change it anytime.

Example:

```
document.querySelector("h1").textContent = "Welcome";
```

DOM updates instantly.

Important Interview Point

DOM is not HTML.

HTML:

```
<h1>Hello</h1>
```

DOM:

```
{  
  tagName: "H1",  
  textContent: "Hello"  
}
```

DOM is an object representation.

Quick Revision

Term	Meaning
DOM	Document Object Model
Structure	Tree
Root	document
Parent	Contains child
Child	Nested element
Sibling	Same parent

Part 5: CSSOM Tree

Just like HTML becomes DOM, CSS becomes CSSOM.

What is CSSOM?

CSSOM stands for:

CSS Object Model

It is a tree representation of CSS rules.

Example

CSS:

```
body {  
  background: black;  
}  
  
h1 {  
  color: white;  
}
```

Browser converts it into:

CSSOM

```
body  
└─ background: black  
  
h1  
└─ color: white
```

Why CSSOM Exists

Browser needs a structured way to understand CSS.

Instead of reading raw text repeatedly.

CSS Parsing Flow

CSS File

↓

Parser

↓

Tokens

↓

CSSOM Tree

Example

```
h1 {  
  color: red;  
}
```

Converted to:

```
{  
  selector: "h1",  
  property: "color",  
  value: "red"  
}
```

Important Point

DOM contains structure.

CSSOM contains styling.

Neither alone can create a webpage.

Quick Revision

Structure	Purpose
DOM	HTML Structure
CSSOM	CSS Styles

Part 6: DOM + CSSOM = Render Tree

Now the magic happens.

What is Render Tree?

Render Tree is a combination of:

DOM Tree

+

CSSOM Tree

=

Render Tree

Why Render Tree?

Browser needs:

Structure + Styling

before displaying anything.

Example

DOM:

body

└─ h1

CSSOM:

h1

└─ color:red

Render Tree:

body

h1

└─ color:red

Visual Flow

HTML

↓

DOM

CSS

↓

CSSOM

DOM + CSSOM

↓

Render Tree

Hidden Elements

Elements with:

`display:none;`

are NOT added to Render Tree.

Example:

```
<h1>Hello</h1>
```

```
<p style="display:none">Hidden</p>
```

Render Tree contains:

h1

Only.

After Render Tree

Browser performs:

Layout

Calculates:

Width

Height

Position

Margin

Padding

Paint

Actually draws:

Text

Colors

Borders

Images

onto the screen.

Complete Rendering Pipeline

HTML

↓

Parsing

↓

Tokenization

↓

DOM Tree

CSS

↓

Parsing

↓

CSSOM Tree

DOM + CSSOM

↓

Render Tree

↓

Layout



Paint



Screen

Part 7: Selection of Elements

JavaScript interacts with webpages through DOM element selection.

Why Select Elements?

To:

- Change text
- Change styles
- Add events
- Create elements
- Delete elements

1. getElementById()

Selects element using ID.

HTML:

```
<h1 id="title">Hello</h1>
```

JS:

```
const heading =  
document.getElementById("title");  
console.log(heading);
```

2. getElementsByClassName()

HTML:

```
<p class="text">One</p>
```

```
<p class="text">Two</p>
```

JS:

```
const items =
```

```
document.getElementsByClassName("text");
```

Returns:

HTMLCollection

3. `getElementsByTagName()`

```
document.getElementsByTagName("p");
```

Returns all p elements.

4. `querySelector()`

Returns first matching element.

```
document.querySelector(".box");
```

```
document.querySelector("#title");
```

```
document.querySelector("p");
```

5. `querySelectorAll()`

Returns all matching elements.

```
document.querySelectorAll(".box");
```

Returns:

NodeList

Difference Between `querySelector` and `querySelectorAll`

<code>querySelector</code>	<code>querySelectorAll</code>
First Match	All Matches

querySelector	querySelectorAll
Single Element	NodeList

Most Commonly Used in Modern JavaScript

`document.querySelector()`

`document.querySelectorAll()`

These are the most popular methods in React, JavaScript, and modern frontend development.

Final DOM Revision Sheet 🚀

Topic	Key Point
Parsing	Browser reads HTML
Tokenization	HTML → Tokens
DOM Tree	HTML structure
CSSOM Tree	CSS structure
Render Tree	DOM + CSSOM
Layout	Calculate positions
Paint	Draw pixels
<code>getElementById</code>	Select by ID
<code>getElementsByClassName</code>	Select by class
<code>getElementsByTagName</code>	Select by tag
<code>querySelector</code>	First match
<code>querySelectorAll</code>	All matches

Browser Rendering Learning Path

HTML

↓

Parsing

↓

Tokenization

↓

DOM Tree

CSS

↓

Parsing

↓

CSSOM Tree

DOM + CSSOM

↓

Render Tree

↓

Layout

↓

Paint

↓

Screen

Then

JavaScript

↓

DOM Selection

↓

DOM Manipulation

This flow is the foundation of the DOM and browser rendering engine, and it is one of the most frequently asked topics in frontend and JavaScript interviews

Dom class 2 Notes:-

Part 1: DOM Tree

The DOM Tree is one of the most important concepts in JavaScript and Web Development.

Before learning how to manipulate elements, we must understand how the browser represents HTML internally.

What is DOM?

DOM stands for:

Document Object Model

The browser converts HTML into a tree-like structure called the DOM Tree.

This tree allows JavaScript to:

- Read elements
- Modify elements
- Add elements
- Remove elements
- Update content dynamically

Why Do We Need DOM?

Suppose we have:

```
<h1>Hello World</h1>
```

HTML itself is just text.

JavaScript cannot directly manipulate this text.

Therefore the browser converts HTML into objects.

HTML

↓

DOM

↓

JavaScript Manipulates DOM

Example

HTML:

```
<html>
<body>
<h1>Hello</h1>
<p>Welcome</p>
</body>
</html>
```

DOM Tree:

Document

```
|
├── html
|
├── body
|
├── h1
|   ├── Hello
|
├── p
|   └── Welcome
```

Understanding Parent, Child and Sibling

Parent Node

```
<body>
<h1>Hello</h1>
</body>
```

Here:

body

is parent of

h1

Child Node

h1

is child of

body

Sibling Nodes

```
<h1>Hello</h1>
```

```
<p>Welcome</p>
```

Both have the same parent.

Therefore:

h1 and p

are siblings.

DOM is Live

JavaScript can modify DOM anytime.

Example:

```
document.querySelector("h1").textContent = "Welcome";
```

Before:

```
<h1>Hello</h1>
```

After:

```
<h1>Welcome</h1>
```

The webpage updates instantly.

Important Interview Point

HTML and DOM are not the same.

HTML:

```
<h1>Hello</h1>
```

DOM:

```
{  
  tagName: "H1",  
  textContent: "Hello"  
}
```

HTML is text.

DOM is an object representation.

Quick Revision

Term	Meaning
DOM	Document Object Model
Structure	Tree
Parent	Contains child
Child	Nested inside parent
Sibling	Same parent
Root Node	document

Part 2: Fetching Elements in DOM

Before modifying elements, we first need to select them.

This process is called:

DOM Selection

or

Fetching Elements

Why Fetch Elements?

To:

- Change text
- Change styles

- Add events
- Remove elements
- Create elements

JavaScript must first locate the element.

1. getElementById()

Selects an element using its ID.

HTML:

```
<h1 id="title">Hello</h1>
```

JavaScript:

```
const heading =  
document.getElementById("title");  
console.log(heading);
```

Output:

```
<h1 id="title">Hello</h1>
```

Important Rule

IDs should be unique.

```
<h1 id="title">Hello</h1>
```

Only one element should have that ID.

2. getElementsByClassName()

Selects all elements with the same class.

HTML:

```
<p class="text">One</p>  
<p class="text">Two</p>  
<p class="text">Three</p>
```

JavaScript:

```
const items =
```

```
document.getElementsByClassName("text");  
console.log(items);
```

Output:

HTMLCollection(3)

3. `getElementsByTagName()`

Selects all elements of a tag.

HTML:

```
<p>One</p>  
<p>Two</p>  
<p>Three</p>
```

JavaScript:

```
const paragraphs =  
document.getElementsByTagName("p");
```

Output:

HTMLCollection(3)

4. `querySelector()`

Returns the first matching element.

HTML:

```
<div class="box"></div>  
<div class="box"></div>
```

JavaScript:

```
const box =  
document.querySelector(".box");
```

Returns:

First .box element only

Examples

Select by ID:

```
document.querySelector("#title");
```

Select by Class:

```
document.querySelector(".box");
```

Select by Tag:

```
document.querySelector("p");
```

5. querySelectorAll()

Returns all matching elements.

HTML:

```
<div class="box"></div>
```

```
<div class="box"></div>
```

```
<div class="box"></div>
```

JavaScript:

```
const boxes =
```

```
document.querySelectorAll(".box");
```

Output:

```
NodeList(3)
```

Difference Between querySelector and querySelectorAll

querySelector	querySelectorAll
First Match	All Matches
Single Element	NodeList
Returns Element	Returns Collection

Modern JavaScript Preference

Most developers use:

`document.querySelector()`
`document.querySelectorAll()`
because they support CSS selectors.

Quick Revision

Method	Purpose
<code>getElementById()</code>	Select by ID
<code>getElementsByClassName()</code>	Select by Class
<code>getElementsByTagName()</code>	Select by Tag
<code>querySelector()</code>	First Match
<code>querySelectorAll()</code>	All Matches

Part 3: innerHTML

One of the most commonly used DOM properties.

What is innerHTML?

innerHTML allows us to:

Read HTML Content

OR

Write HTML Content

inside an element.

Example

HTML:

```
<div id="box">
```

```
<h1>Hello</h1>
```

```
</div>
```

JavaScript:

```
const box =
```

```
document.getElementById("box");  
console.log(box.innerHTML);
```

Output:

```
<h1>Hello</h1>
```

Updating innerHTML

```
box.innerHTML =
```

```
"<h2>Welcome</h2>";
```

Result:

```
<div id="box">
```

```
<h2>Welcome</h2>
```

```
</div>
```

Adding Multiple Elements

```
box.innerHTML = `
```

```
<h1>Hello</h1>
```

```
<p>Welcome</p>
```

```
<button>Click Me</button>
```

```
`;  
;
```

Important Point

innerHTML understands HTML tags.

Example:

```
box.innerHTML =
```

```
"<strong>Hello</strong>";
```

Output:

Hello

displayed in bold.

Security Warning

Never insert untrusted user input using innerHTML.

Example:

```
box.innerHTML = userInput;
```

Can lead to:

XSS Attacks

Quick Revision

Feature	innerHTML
Reads HTML	Yes
Writes HTML	Yes
Understands Tags	Yes
Security Risk	Yes

Part 4: textContent

Another very important DOM property.

What is textContent?

textContent reads or updates only text.

It ignores HTML tags.

Example

HTML:

```
<div id="box">
```

```
<h1>Hello</h1>
```

```
</div>
```

JavaScript:

```
const box =
```

```
document.getElementById("box");
```

```
console.log(box.textContent);
```

Output:

Hello

Notice:

```
<h1>
```

is not returned.

Only text is returned.

Updating textContent

```
box.textContent =
```

```
"Welcome to JavaScript";
```

Result:

```
<div id="box">
```

Welcome to JavaScript

```
</div>
```

Difference Example

Using innerHTML:

```
box.innerHTML =
```

```
"<strong>Hello</strong>";
```

Output:

Hello

Using textContent:

```
box.textContent =
```

```
"<strong>Hello</strong>";
```

Output:

```
<strong>Hello</strong>
```

Displayed as plain text.

Why `textContent` is Safer?

Because it treats everything as text.

No HTML execution.

No script execution.

Less security risk.

`innerHTML` vs `textContent`

<code>innerHTML</code>	<code>textContent</code>
Reads HTML	Reads Text
Writes HTML	Writes Text
Understands Tags	Ignores Tags
Less Secure	More Secure

Part 5: `classList` 🎨

`classList` is used to manage CSS classes dynamically.

This is heavily used in:

- DOM Manipulation
- Dark Mode
- Modals
- Dropdowns
- Accordions
- React Projects

What is `classList`?

`classList` provides methods to:

Add Classes

Remove Classes

Toggle Classes

Check Classes

Example

HTML:

```
<div id="box"></div>
```

JavaScript:

```
const box =  
document.getElementById("box");
```

1. `classList.add()`

Adds a class.

```
box.classList.add("active");
```

Result:

```
<div id="box" class="active"></div>
```

Adding Multiple Classes

```
box.classList.add(  
"active",  
"visible",  
"dark"  
);
```

2. `classList.remove()`

Removes a class.

```
box.classList.remove("active");
```

Example

Before:

```
<div class="active"></div>
```

After:

```
<div></div>
```

3. `classList.toggle()`

Most commonly used.

If class exists:

Remove it

If class doesn't exist:

Add it

Example

```
box.classList.toggle("dark");
```

Used for:

Dark Mode

Mobile Menu

Sidebar

Modal

Dark Mode Example

```
button.addEventListener("click", () => {  
  document.body.classList.toggle("dark");  
});
```

4. `classList.contains()`

Checks whether a class exists.

```
box.classList.contains("active");
```

Output:

true

or

false

Example

```
if(box.classList.contains("active")){  
  console.log("Class Found");  
}
```

Complete Example

HTML:

```
<button id="btn">Toggle</button>
```

```
<div id="box">
```

Content

```
</div>
```

CSS:

```
.hidden{
```

```
  display:none;
```

```
}
```

JavaScript:

```
const btn =
```

```
document.getElementById("btn");
```

```
const box =
```

```
document.getElementById("box");
```

```
btn.addEventListener("click", () => {
```

```
  box.classList.toggle("hidden");
```

```
});
```

Every click:

Show

Hide

Show

Hide

Quick Revision

Method	Purpose
add()	Add class
remove()	Remove class
toggle()	Add/Remove automatically
contains()	Check class

Final DOM Manipulation Revision Sheet 🚀

Topic	Key Point
DOM Tree	Object representation of HTML
getElementById()	Select by ID
getElementsByClassName()	Select by Class
getElementsByTagName()	Select by Tag
querySelector()	First matching element
querySelectorAll()	All matching elements
innerHTML	Reads/Writes HTML
textContent	Reads/Writes Text
classList.add()	Add class
classList.remove()	Remove class
classList.toggle()	Toggle class
classList.contains()	Check class

Learning Flow

DOM Tree



Fetching Elements



innerHTML



textContent



classList



DOM Manipulation



Interactive Web Pages

Master these five topics thoroughly before moving to Events and Event Listeners, because almost every DOM project uses these concepts daily.